

Ouvidoria Inteligente

Triagem Semântica de Manifestações Cidadãs | Desafio Prático — NLP Aplicado

Kaike Garcia · Pedro Jorge Fernandes · Samuel Ramos

github.com/Samuelptdd/Desafio_NLP_Aplicado | setembro de 2026

1. O problema e o que foi construído

A Ouvidoria Geral recebe cerca de 4.000 manifestações por mês em texto livre e faz a triagem por palavra-chave. Isso gera três falhas: duplicatas semânticas não são detectadas, temas relacionados não são agrupados, e manifestações longas perdem contexto ao serem indexadas como bloco único.

Construímos um protótipo que substitui a comparação entre palavras pela comparação entre **significados**. O corpus tem 40 manifestações (86 a 712 caracteres, cinco delas acima de 500), distribuídas em cinco categorias oficiais.

O exemplo que resume o problema são as manifestações M003 e M017. As duas relatam o mesmo buraco, na mesma avenida:

M003: "Buraco enorme na Av. Brasil, perto do número 1200. Vários carros já estouraram pneu..."

M017: "Asfalto todo esburacado da avenida principal. Não dá para passar sem cair em uma cratera..."

Elas **não compartilham nenhuma palavra de conteúdo**. Medimos: o TF-IDF entre elas é **0.000**. Nenhum ajuste da busca textual encontraria esse par, porque a informação não está nas palavras. O embedding dá 0.594 e as aproxima. É esse salto que justifica trocar a tecnologia em vez de afinar a busca antiga.

2. Decisões técnicas

2.1 Corpus único para as quatro entregas

O repositório continha dois arquivos de manifestações, e as entregas começaram usando arquivos diferentes. Padronizamos no `manifestacoes_exemplo.json` depois de constatar que o outro tem cinco registros — justamente os que o enunciado cita — com 24 a 44 caracteres, abaixo do mínimo de 50 que a seção 2 define. Medido com o mesmo modelo, o par M003 × M017 cai de 0.594 para 0.287 naquele arquivo: 27 caracteres não dão contexto suficiente para um embedding.

2.2 Modelo de embedding

Adotamos o `paraphrase-multilingual-MiniLM-L12-v2` nas quatro entregas, por ser multilíngue, rápido e suficiente para o corpus. O app permite trocar em tempo real para o `mpnet-base-v2` (768 dimensões, separa melhor os pares) e para o `multilingual-e5-small` (otimizado para busca). Usar o mesmo modelo em todas as entregas foi uma decisão deliberada: modelos diferentes produzem escalas diferentes, e os números deixariam de ser comparáveis entre si.

2.3 Limiar de duplicata: calibrado, não herdado

O enunciado sugere 0.85. Implementamos com esse valor e a função encontrou **1 dos 8 pares** de duplicata — recall de 12,5%. Em vez de aceitar ou descartar o número, medimos as duas distribuições e calibramos:

Critério	VP	FP	FN	Precisão	Recall	F1
limiar 0.85, como sugerido	1	0	7	1.000	0.125	0.222
limiar 0.70, calibrado	5	1	3	0.833	0.625	0.714
triagem em três faixas	8	—	0	—	1.000	—

A terceira linha é a decisão de projeto mais importante do trabalho. Mesmo com 0.70, dois pares continuavam escapando — justamente os dois que o enunciado destaca. O problema não era o valor do corte, era o **formato**: um limiar único obriga o sistema a escolher entre agrupar e descartar. Adotando as três faixas de cor que o próprio enunciado define para a Entrega 4 (verde acima de 0.7, amarelo acima de 0.5), nenhuma das oito duplicatas se perde: cinco são agrupadas automaticamente e três vão para conferência humana.

A assimetria dos erros sustenta essa escolha. Um falso positivo arquiva a reclamação de um cidadão como repetida e **ele nunca recebe resposta**. Um falso negativo gera dois protocolos para o mesmo buraco: é desperdício, mas ninguém é prejudicado. Por isso o sistema é de **triagem**, não de decisão — ele ordena o trabalho, um servidor

confirma.

2.4 Gabarito construído por leitura

O dataset não vem com ground truth, e a análise de falsos positivos/negativos exigida pelo enunciado depende dele. Identificamos os oito pares manualmente, com um critério explícito e conferível: **mesmo local nomeado + mesmo problema**. Está documentado par a par em dados/gabarito_exemplo.json, e qualquer discordância pode ser corrigida editando o arquivo — os notebooks recalculam tudo.

3. Dificuldades encontradas

3.1 O truncamento silencioso em 128 tokens

O modelo lê no máximo 128 tokens, cerca de 450 caracteres em português, e **descarta o resto sem gerar erro nem aviso**. Cinco manifestações do corpus são afetadas; a maior perde 23% do texto. Foi a dificuldade mais difícil de perceber justamente por ser silenciosa: o código roda, os números saem, e nada indica que metade do texto não foi lida.

O efeito é visível nos resultados. As três manifestações longas do corpus começam com fórmulas parecidas (“Venho por meio desta...”, “Venho denunciar...”). Truncadas, passam a ser representadas majoritariamente pela **abertura formal**, e pares delas sobre assuntos completamente diferentes — drenagem, furtos, poluição — pontuaram acima de duplicatas verdadeiras. O embedding estava medindo estilo em vez de assunto.

3.2 Pedir overlap não garante overlap

Na Entrega 3, comparamos duas configurações de chunking e concluímos inicialmente que o *overlap* maior havia melhorado a coesão entre chunks. A revisão mostrou que as duas variáveis — `chunk_size` e `chunk_overlap` — haviam mudado ao mesmo tempo, então o experimento não conseguia separar um efeito do outro. Refizemos com o tamanho fixo:

Config.	chunk_size	overlap pedido	pares com overlap real	Coesão
1	200	40	0 de 22	0.3278
2	300	80	0 de 13	0.4751
3	300	0	0 de 13	0.4751
4	300	150	6 de 15 (52,9 car.)	0.5670

As Configurações 2 e 3 têm coesão **idêntica**: pedir overlap de 80 caracteres não produziu sobreposição nenhuma. O `RecursiveCharacterTextSplitter` corta em **separadores**, não no meio da frase, e as frases destes textos têm de 107 a 249 caracteres — um orçamento de 80 não comporta uma frase, então não há o que repetir.

O conceito estava correto: com overlap de 150, maior que a frase típica, a coesão sobe de 0.4751 para **0.5670**. O que estava errado era a atribuição. É o tipo de erro mais perigoso, porque o parâmetro é aceito sem reclamação e simplesmente não faz efeito.

3.3 Reprodutibilidade

O notebook da Entrega 1 escolhia o corpus automaticamente e preferia o arquivo que, no repositório, não era o usado para gerar as saídas salvas. Quem clonasse e reexecutasse obteria números diferentes dos publicados, e a discussão passaria a citar trechos inexistentes. Corrigimos fixando o corpus. O aprendizado prático: **em notebook, escolha automática de arquivo é uma armadilha** — o que se ganha em conveniência se perde em reprodutibilidade.

4. Aprendizados

1. Similaridade de cosseno não é porcentagem de parecença.

Foi o erro conceitual que custou mais tempo. A escala depende do modelo, do idioma e de quanto o corpus é homogêneo. Um limiar transplantado de outro contexto não funciona: 0.85 encontrou 1 das 8 duplicatas. Limiar precisa ser calibrado no próprio corpus e recalibrado quando o corpus muda.

2. Escolher limiar é decisão de processo, não de estatística.

A mesma curva de precisão e recall sustenta 0.70 ou 0.74. O que decide é se existe revisão humana antes de arquivar. O F1 trata os dois erros como igualmente ruins; numa Ouvidoria eles não são.

3. Embeddings resolvem o que palavra-chave não resolve — e o inverso também vale.

O par M003 × M017 tem TF-IDF 0.000 e embedding 0.594. Mas o embedding também dá 0.2 a 0.3 para pares sem nenhuma relação, o que obriga a escolher o limiar com cuidado. Nenhuma das duas representações é suficiente sozinha; o que muda é o tipo de erro que cada uma comete.

4. Medir é diferente de rodar.

Os dois achados mais valiosos do trabalho — o truncamento silencioso e o overlap que não acontecia — não apareceram em mensagem de erro nenhuma. Apareceram quando fomos verificar, com número, se o que pedimos à ferramenta era o que ela estava de fato fazendo.

5. O espaço vetorial reflete as categorias, mas não as substitui.

Manifestações da mesma categoria oficial são 76% mais parecidas entre si (0.4165) do que com as de outras categorias (0.2361) — o modelo captou a organização temática sem nunca ter visto os rótulos. Mas o coeficiente de silhueta é 0.120, indicando fronteiras difusas. As categorias são administrativas e o espaço é semântico: “iluminação numa travessa escura” pode ser infraestrutura ou segurança. E os agrupamentos que o modelo forma são mais finos que as categorias — dentro de saúde ele separa “falta de medicamento” de “demora no atendimento”.

5. Limitações declaradas

- **Custo quadrático.** A matriz completa é $O(n^2)$: 780 comparações para 40 manifestações, mas cerca de **8 milhões** para as 4.000 por mês reais. Em produção seria preciso busca aproximada de vizinhos (FAISS, HNSW) em vez de matriz completa.
- **Limiar calibrado no conjunto de avaliação.** Com 8 duplicatas não há como separar treino e teste de forma útil. O F1 de 0.714 é otimista. Num sistema real, a calibração seria feita sobre manifestações já revisadas por servidores.
- **Gabarito é interpretação nossa.** O critério adotado é explícito e conferível, mas alguns pares são discutíveis — M004 e M021 são ambos descarte irregular de lixo, marcados como não-duplicata apenas porque citam ruas diferentes.
- **Truncamento não foi resolvido, apenas medido.** O chunking trata o problema na Entrega 3, mas as Entregas 1, 2 e a busca do app ainda indexam o texto inteiro e sofrem o corte.
- **Corpus pequeno e sintético.** 40 manifestações, 8 duplicatas. Uma duplicata a mais ou a menos move o F1 em cerca de 0,06.

6. Resultado e próximos passos

O protótipo atende aos três problemas do enunciado. Detecta duplicatas que a busca por palavra-chave não alcança; agrupa temas relacionados num espaço vetorial que reflete a organização da Ouvidoria; e trata textos longos com chunking, com a ressalva de que o parâmetro de sobreposição precisa ser maior que a unidade de corte para ter efeito.

A recomendação de uso é como **triagem assistida**, com a faixa intermediária encaminhada para conferência humana. Para escalar às 4.000 manifestações mensais, os próximos passos seriam substituir a matriz completa por busca aproximada de vizinhos, adotar um modelo com janela maior que 128 tokens, e recalibrar o limiar periodicamente a partir do histórico de decisões dos servidores.